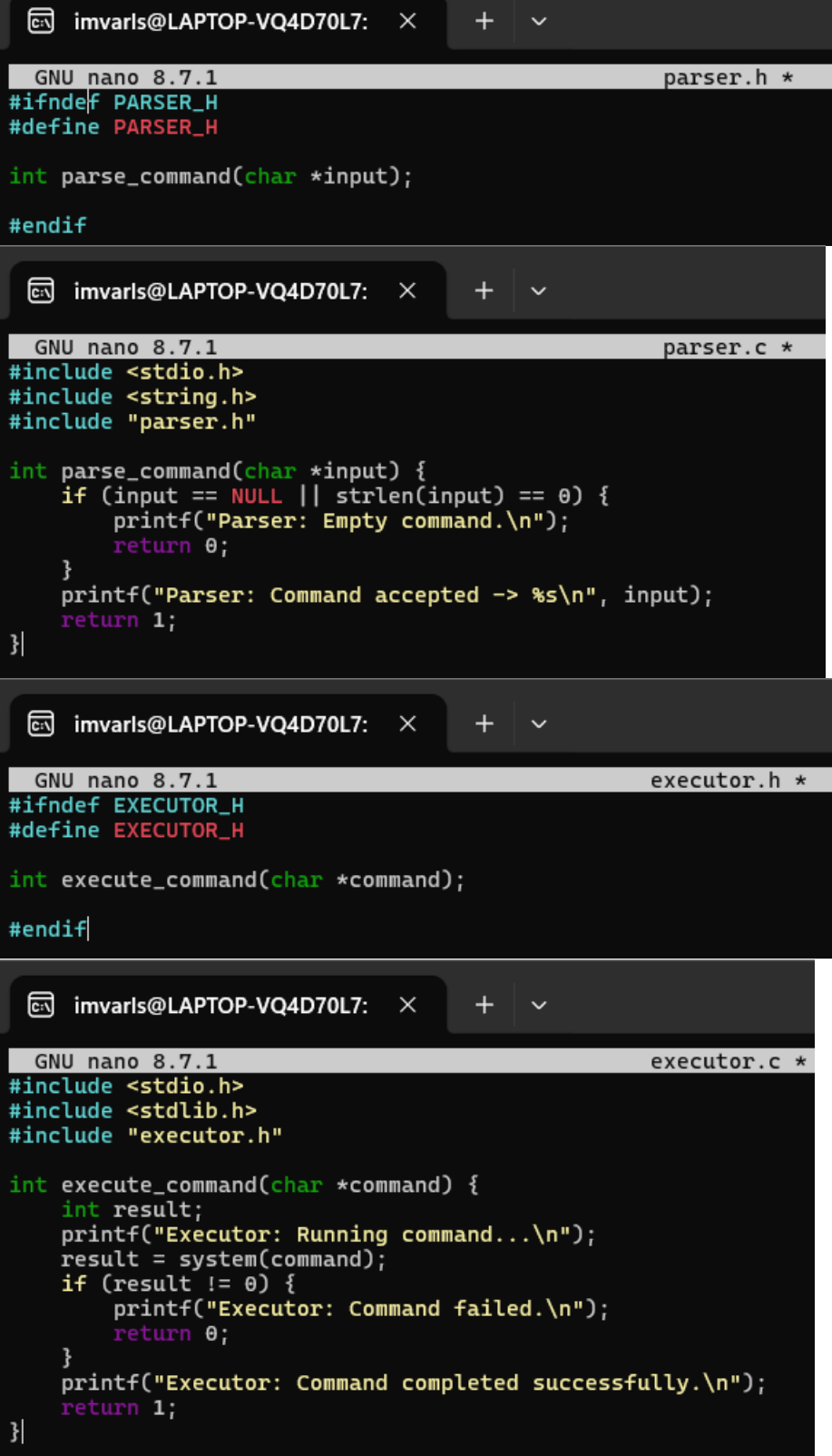


Skill Week-11

1. To Connect Modules, Validate Interfaces, Test End-to-End Execution, Handle Errors, Improve Reliability, Refactor Components



```
imvarls@LAPTOP-VQ4D70L7: × + ▾
GNU nano 8.7.1 parser.h *
#ifndef PARSER_H
#define PARSER_H

int parse_command(char *input);

#endif

imvarls@LAPTOP-VQ4D70L7: × + ▾
GNU nano 8.7.1 parser.c *
#include <stdio.h>
#include <string.h>
#include "parser.h"

int parse_command(char *input) {
    if (input == NULL || strlen(input) == 0) {
        printf("Parser: Empty command.\n");
        return 0;
    }
    printf("Parser: Command accepted -> %s\n", input);
    return 1;
}

imvarls@LAPTOP-VQ4D70L7: × + ▾
GNU nano 8.7.1 executor.h *
#ifndef EXECUTOR_H
#define EXECUTOR_H

int execute_command(char *command);

#endif

imvarls@LAPTOP-VQ4D70L7: × + ▾
GNU nano 8.7.1 executor.c *
#include <stdio.h>
#include <stdlib.h>
#include "executor.h"

int execute_command(char *command) {
    int result;
    printf("Executor: Running command...\n");
    result = system(command);
    if (result != 0) {
        printf("Executor: Command failed.\n");
        return 0;
    }
    printf("Executor: Command completed successfully.\n");
    return 1;
}
```

```
imvarls@LAPTOP-VQ4D70L7: × + ▾
GNU nano 8.7.1 main.c *
#include <stdio.h>
#include <string.h>
#include "parser.h"
#include "executor.h"

int main() {
    char command[100];

    printf("Enter command: ");
    fgets(command, sizeof(command), stdin);
    command[strcspn(command, "\n")] = '\0';

    if (!parse_command(command)) {
        printf("Main: Invalid command.\n");
        return 1;
    }

    if (!execute_command(command)) {
        printf("Main: Execution failed.\n");
        return 1;
    }

    printf("Main: End-to-end execution successful.\n");
    return 0;
}
```

Output:



imvarls@LAPTOP-VQ4D70L7: ×

+

v

```
imvarls@LAPTOP-VQ4D70L7:~$ gcc main.c parser.c executor.c -o modular_shell
```

```
imvarls@LAPTOP-VQ4D70L7:~$ ./modular_shell
```

```
Enter command: ls
```

```
Parser: Command accepted -> ls
```

```
Executor: Running command...
```

```
File.txt
```

```
File1.tx
```

```
File1.txt
```

```
File1.txt.c
```

```
Linux-Dynamic-Memory-Allocation-and-Memory-Monitoring-System
```

```
OSSP
```

```
append.txt
```

```
append_redirect
```

```
append_redirect.c
```

```
background_jobs
```

```
background_jobs.c
```

```
client
```

```
client.c
```

```
combined.txt
```

```
combined_redirect
```

```
combined_redirect.c
```

```
command_history
```

```
command_history.c
```

```
command_parser
```

```
command_parser.c
```

```
copy
```

```
copy.c
```

```
copy.txt
```

```
copy2.txt
```

```
copy_file.c
```

```
copy_stdio
```

```
copy_stdio.c
```

```
copy_syscalls
```

```
copy_syscalls.c
```

```
deadlock
```

```
deadlock.c
```

```
demo
```

```
demo.c
```

```
directory
```

```
directory.c
```

```
dynamic_memory
```

```
dynamic_memory.c
```

```
error.txt
```

```
executor.c
```

```
executor.h
```

```
export
```

```
export.c
```

```
file.c
```

```
file.java
```

```
file2.c
```

```
filename
```

```
filename.c
```

```
filename.c.save
```

```
first.c.save
```

```
first.c.save.1
```

```
Executor: Command completed successfully.
```

```
Main: End-to-end execution successful.
```

```
imvarls@LAPTOP-VQ4D70L7:~$ ./modular_shell
```

```
Enter command: invalid_command_xyz
```

```
Parser: Command accepted -> invalid_command_xyz
```

```
Executor: Running command...
```

```
sh: 1: invalid_command_xyz: not found
```

```
Executor: Command failed.
```

```
Main: Execution failed.
```

```
imvarls@LAPTOP-VQ4D70L7:~$ |
```

```
foreground_switch      producer_consumer
foreground_switch.c    producer_consumer.c
hardlink.txt           race_condition
input.txt              race_condition.c
input_redirect         redirection
input_redirect.c      redirection.c
job_table              resume_job
job_table.c            resume_job.c
list_jobs              server
list_jobs.c            server.c
main.c                 sever
map_input.txt          shell
memory                 shell.c
memory.c               signal_action
memory_address         signal_action.c
memory_address.c      signal_behavior
memory_leak            signal_behavior.c
memory_leak.c          signal_forward
memory_maps            signal_forward.c
memory_maps.c          sigtspt_job
mmap_file              sigtstp_job
mmap_file.c            sigtstp_job.c
mmap_input.txt         singlepipe
mmutex_counter         singlepipe.c
multiple_pipe          snap
multiple_pipe.c        state.txt
mutex_counter          stderr_redirect
mutex_counter.c        stderr_redirect.c
myshell                symlink.txt
myshell.c              task2
navigation              task2.c
navigation.c           week1
originaal.txt          week1.c
output.txt             week4
output_redirect        week4.c
output_redirect.c      week5
parser.c               week5.c
parser.h               week5task2
pipeline               week5task2.c
pipeline.c             week6client
practical1             week6client.c
practical1.c           week6server
process                week6server.c
process.c              week6task2
process_group          week6task2.c
process_group.c        weektask2
process_state          write
process_state.c        write.c
processstate.c         zombie
                       zombie.c
```

2. To Detect Runtime Errors, Generate Messages, Log Failures, Handle Invalid Syntax, Recover Gracefully, Test Error Scenarios

```
imvarls@LAPTOP-VQ4D70L7: × + v
GNU nano 8.7.1 error_handler.h *
#ifndef ERROR_HANDLER_H
#define ERROR_HANDLER_H

void log_error(const char *message);

#endif
```

```
imvarls@LAPTOP-VQ4D70L7: × + v
GNU nano 8.7.1 error_handler.c *
#include <stdio.h>
#include "error_handler.h"

void log_error(const char *message) {
    FILE *file = fopen("error.log", "a");
    if (file == NULL) {
        printf("Unable to open error log.\n");
        return;
    }
    fprintf(file, "ERROR: %s\n", message);
    fclose(file);
    printf("ERROR: %s\n", message);
}
```

```
imvarls@LAPTOP-VQ4D70L7: × + v
GNU nano 8.7.1 error_test.c *
#include <stdio.h>
#include <string.h>
#include <stdlib.h>
#include "error_handler.h"

int main() {
    char command[100];

    printf("Enter command: ");
    fgets(command, sizeof(command), stdin);
    command[strcspn(command, "\n")] = '\0';

    if (strlen(command) == 0) {
        log_error("Empty command entered.");
        printf("Recovering: Please enter a command next time.\n");
        return 1;
    }

    if (strstr(command, "&&&") != NULL) {
        log_error("Invalid command syntax.");
        printf("Recovering: Invalid syntax ignored.\n");
        return 1;
    }

    printf("Executing command...\n");
    int result = system(command);
    if (result != 0) {
        log_error("Command execution failed.");
        printf("Recovering: Shell remains active.\n");
        return 1;
    }

    printf("Command executed successfully.\n");
    return 0;
}
```

Output:

```
imvarls@LAPTOP-VQ4D70L7:~$ nano error_handler.h
imvarls@LAPTOP-VQ4D70L7:~$ nano error_handler.c
imvarls@LAPTOP-VQ4D70L7:~$ nano error_test.c
imvarls@LAPTOP-VQ4D70L7:~$
imvarls@LAPTOP-VQ4D70L7:~$ gcc error_test.c error_handler.c -o error_test
imvarls@LAPTOP-VQ4D70L7:~$ ./error_test
Enter command: pwd
Executing command...
/home/imvarls
Command executed successfully.
imvarls@LAPTOP-VQ4D70L7:~$
imvarls@LAPTOP-VQ4D70L7:~$ |
```

3. To Run Valgrind, Identify Leaks, Fix Memory Errors, Release Resources, Verify Cleanup, Document Findings

```
GNU nano 8.7.1 memory_task.c *
#include <stdio.h>
#include <stdlib.h>

int main() {
    int *arr;
    arr = malloc(5 * sizeof(int));
    if (arr == NULL) {
        printf("Memory allocation failed.\n");
        return 1;
    }

    for (int i = 0; i < 5; i++) {
        arr[i] = i + 1;
    }

    printf("Array values: ");
    for (int i = 0; i < 5; i++) {
        printf("%d ", arr[i]);
    }
    printf("\n");

    free(arr);
    arr = NULL;

    printf("Memory released successfully.\n");
    return 0;
}
```

Output:

```
imvarls@LAPTOP-VQ4D70L7:~$ nano memory_task.c
imvarls@LAPTOP-VQ4D70L7:~$ gcc memory_task.c -o memory_task -g
imvarls@LAPTOP-VQ4D70L7:~$ ./memory_task
Array values: 1 2 3 4 5
Memory released successfully.
imvarls@LAPTOP-VQ4D70L7:~$ |
```

